

Security & Detection Engineer (Person 3) - Deployment Pack

This document serves as the complete reference for **Person 3**, covering implementation goals, team dependencies, and the technical data contract required from Person 1.

🚩 1. Core Responsibilities & Tasks

Your mission is to transition the security logic from a prototype to a production-ready intelligence module.

Key Implementation Goals:

- ☐ **State Persistence:** Replace in-memory `Map` in `SecurityEngine.js` with database/shared-cache lookups.
- ☐ **Threshold Externalization:** Move security constants (DDoS limits, CPU thresholds) to `.env`.
- ☐ **Response Automation:** Ensure `Critical` alerts trigger a `Blocked` status in the `Device` collection automatically.
- ☐ **Compound Threat Detection:** Implement logic for detecting sequences of anomalies (e.g., Anomaly + Unauthorized Access).

👤 2. Team Dependencies

Your work sits at the intersection of data production and visualization.

- From Person 1 (IoT/Simulator):** You require consistent telemetry fields and specific attack simulation sequences to test your engine.
- From Person 2 (Backend Architect):** You require the `/telemetry` route to correctly execute your `analyzeTelemetry` logic and provide access to the DB.
- To Person 4 (Dashboard Developer):** You must provide structured `Incident` payloads that the React Dashboard can render as real-time alerts.

📄 3. IoT Telemetry Data Contract

To be provided to Person 1.

Every packet sent by the IoT Simulator **MUST** follow this JSON structure for your engine to process it correctly.

Telemetry Schema

```
{
  "deviceId": "ESP32-001",
  "timestamp": "2024-03-05T22:50:00Z",
  "type": "heartbeat",
  "data": {
    "cpuUsage": 12.5,
    "ramUsage": 45.2,
    "packetFrequency": 1,
```

```
"loginStatus": "SUCCESS",
"authHeader": "Bearer eyJhbGci...\"",
"sensorData": {
  "temperature": 22.5,
  "humidity": 45.0
}
}
```

Field Constraints

- **type options:** heartbeat , login , traffic_spike .
- **loginStatus options:** SUCCESS , FAIL .
- **cpuUsage** : Range 0-100.
- **sensorData.temperature** : Range -10 to 80.



4. Attack Simulation Protocols

Person 1 should implement these behaviors to verify Person 3's logic:

1. **Protocol BF (Brute Force):** 6 failed login packets in 10s.
2. **Protocol DS (DDoS Stress):** 20+ packets in < 2s with high frequency.
3. **Protocol SM (Sensor Manipulation):** Rapid temperature jump (>15°C) in a single interval.
4. **Protocol RD (Rogue Device):** Mismatched `deviceId` in JSON body vs. Authentication token.



5. Verification Plan

- **Automated:** Use `test_security_rules.js` to send malicious payloads and verify Incident creation.
- **Manual:** Trigger a simulator attack and verify the dashboard red alerts appear in < 2s.